

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো
(Prof. Chester Rebeiro)
কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering),
ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাস
(Indian Institute of Technology, Madras)
সপ্তাহ – 08
বক্তৃতা – 37
অপারেটিং সিস্টেম নিরাপত্তা - বাফার ওভারফ্লো
(Operating System Security -Buffer Overflows)

হ্যাঁ। এই ভিডিওতে, আমরা Buffer Overflow সম্পর্কে কথা বলব। মূলত, Buffer Overflow সিস্টেমের মধ্যে একটি ঝুঁকি (vulnerability) এবং এটি কেবলমাত্র অপারেটিং সিস্টেমের মধ্যে সীমাবদ্ধ নয়, তবে এটি সিস্টেমে চালিত যেকোনো অ্যাপ্লিকেশনের সাথে সম্পর্কিত হতে পারে। এখন Buffer Overflow হল একটি vulnerability যা malicious অ্যাপ্লিকেশনগুলিকে সিস্টেমে প্রবেশ করার অনুমতি দেয়, যদিও তাদের কোনো বৈধ access নেই। মূলত এটি সিস্টেমের মধ্যে অননুমোদিত (unauthorized) access এর অনুমতি দেবে। সুতরাং আসুন এই বিশেষ বক্তৃতায় Buffer Overflow দেখি।

(স্লাইড সময় : 01:02 দেখুন)

সুতরাং, যখন আমরা দেখি কিভাবে একটি অননুমোদিত user বা একটি অননুমোদিত attacker সিস্টেমের access লাভ করতে পারে তাই আমরা দেখতে পাই যে এটি কেবল ত্রুটির উপস্থিতিতে হয়। একটি সিস্টেমে দুই ধরনের ত্রুটি হতে পারে। এই বিশেষ ক্ষেত্রে বা অ্যাপ্লিকেশন বা অপারেটিং সিস্টেমের মধ্যে উপস্থিত একটি bug হতে পারে বা এটি human factor এর কারণে ত্রুটিগুলি থাকতে পারে। উদাহরণস্বরূপ, যখন আমরা এমন

internet browse করি যেখানে আমরা এমন অনেক web page খুলি এবং আমাদেরকে নির্দিষ্ট কিছু জিনিসগুলিতে ক্লিক করতে অনুরোধ করা হয় যা একটি malicious website হতে পারে এবং এর ফলে malicious অ্যাপ্লিকেশনগুলি সিস্টেমের মধ্যে ডাউনলোড করা হতে পারে। অপারেটিং সিস্টেমের সাথে সম্পর্কিত আরও একটি ত্রুটি হল যখন অপারেটিং সিস্টেম কোডে bug থাকে। এখন, আধুনিক দিনের অপারেটিং সিস্টেমগুলি বিশেষ করে আমরা desktop এবং server গুলিতে সাধারণত যেগুলি ব্যবহার করি তা অত্যন্ত বড় আকারের কোড। উদাহরণস্বরূপ, বর্তমান লিনাক্স কার্নেলটি দশ মিলিয়ন লাইনের কোডের উপরে রয়েছে এবং এই কোডগুলো অবশ্যই প্রোগ্রামারদের দ্বারা লিখিত এবং অসংখ্য bug থাকতে পারে। সুতরাং, এই bug সনাক্ত করা খুব সহজ নয়; যাইহোক, যদি কোন আক্রমণকারী দেখতে চান, তবে সে এই ধরনের একটি bug খুঁজে পেতে পারে এবং তারপর OS এর access লাভের জন্য অপারেটিং সিস্টেমে এই bug টি ব্যবহার করতে পারে। এবং যেহেতু আপনি জানেন যে একবার attacker OS- এ প্রবেশ করতে পারলে তিনি বিভিন্ন জিনিসগুলি করতে সক্ষম হবেন যেমন তিনি অপারেটিং সিস্টেম কোডের বিভিন্ন element গুলি চালাতে সক্ষম হবেন, সেটি সিস্টেমের সমস্ত resource কে নিয়ন্ত্রণ করতে পারে, সেগুলি এছাড়াও কোন user সিস্টেমে execute করবে তা নিয়ন্ত্রণ করবে। সুতরাং, অপারেটিং সিস্টেমের একটি bug এর মাধ্যমে একটি অননুমোদিত user access অত্যন্ত জটিল সমস্যা।

(স্লাইড সময় : 03:20 দেখুন)

অপারেটিং সিস্টেমে অনেকগুলি bug রয়েছে attacker রা অননুমোদিত access পাওয়ার জন্য সুযোগ নেয়। এখানে তাদের কিছু তালিকা দেওয়া হোল। সুতরাং, Buffer overflow হতে পারে প্রোগ্রামের stack বা অপারেটিং সিস্টেমের মধ্যে, heap এ, যাকে Return-to-libc attack বলা হয়। এখানে দ্বিগুণ মুক্ত হতে পারে, মূলত এটি ঘটে যখন একটি মেমরি অবস্থান যা dynamic ভাবে একটি malloc এর মত কিছু দ্বারা বরাদ্দ করা হয় একাধিকবার মুক্ত হতে পারে, integer overflow bug এবং format string bug হতে পারে। সুতরাং সিস্টেমের মধ্যে বিভিন্ন bug এর সুযোগ নিয়ে attacker সিস্টেমে ঢুকতে পারে। তাই, আজ আমরা যা দেখব তা হল stack এর bug এবং Return-to-libc attack আক্রমণ হিসাবে পরিচিত কিছু যা মূলত stack এর Buffer overflow attack এর একটি বিকল্প।

(স্লাইড সময় : 04:22 দেখুন)

সুতরাং, stack এর মধ্যে buffer overflow কিভাবে কাজ করে তা বোঝার জন্য, প্রথমে আমাদের জানতে হবে কিভাবে একটি stack পরিচালিত হয়। সুতরাং আসুন দেখি কিভাবে একটি প্রসেস এর user এর stack পরিচালিত হয়।

(স্লাইড সময় : 04:38 দেখুন)

সুতরাং আসুন এই খুব সাধারণ উদাহরণটি নেওয়া যাক, যার দুটি ফাংশন main এবং এই main থেকে অন্য একটি ফাংশনকে invoke করা হচ্ছে parameter 1, 2 এবং 3 নিয়ে। এবং এই ফাংশনটি দুটি buffer allocate করে - 5 বাইটের buffer 1 এবং 10 বাইটের buffer 2। সুতরাং, আমরা জানি যখন আমরা এই প্রোগ্রামটি সিস্টেমে execute করি হয়, তখন অপারেটিং সিস্টেম বিভিন্ন বিষয় যেমন text বা এই বিশেষ প্রোগ্রামের বিভিন্ন instructions, data section, heap এবং stack তৈরি করে। বিশেষত stack একটি ফাংশন থেকে অন্য ফাংশনে প্যারামিটার পাঠাবার জন্য এবং local variable store করার জন্য ব্যবহৃত হয়। তাই আসুন দেখি কিভাবে এই বিশেষ উদাহরণে stack কিভাবে ব্যবহার করা হয়। সুতরাং আসুন আমরা বলি এই stack main ফাংশন execute করার সময় ব্যবহার হয়। এখন main এখানে এই ফাংশন 1, 2 এবং 3 কে call করতে চায় আর এটি stack এর উপর কিছু push করে। সুতরাং, কি stack এর সম্মুখের দিকে push করা হয় এখন দেখতে হবে। সুতরাং, প্রথমে 3 টি parameter, 1, 2 এবং 3, যা main থেকে ফাংশনটি pass করা হয় stack এ push করা হয়। তারপর return address stack এ push করা হয়। সুতরাং, এই return address টি এই ফাংশন call এর পরের instruction point করবে। X86 space processor এ ফাংশন invoke করার জন্য আমরা জানি যে, call নির্দেশটি ব্যবহৃত হয়। তাই return address ফাংশান invocation এর পরবর্তী instruction নির্দেশ করবে। সুতরাং, return address এর পরে previous frame pointer হিসাবে পরিচিত কিছু stack এ push করা হয়, এই frame pointer main ফাংশনটির frame কে পয়েন্ট করে। সুতরাং, এই frame যা ফাংশন execute করার সময় ব্যবহৃত হয়, এবং যখন main execute করা হয় তখন এই ফ্রেম ব্যবহার করা হয়। এখন previous frame pointer ব্যবহার করার পরে, ফাংশনে সংজ্ঞায়িত local variable allocate করা হয়। এই ক্ষেত্রে, আমাদের দুইটি character array রয়েছে যার একটির size 5 byte এবং অন্যটির 10 byte। তাই এই সব ছাড়াও, আমরা এই stack pointer পরিচালনার জন্য দুইটি CPU register ব্যবহার করি, একটি সাধারণত Intel x86 মধ্যে register bp যা frame pointer এবং অন্যটি stack pointer পয়েন্টার বা sp। এখন frame pointer বর্তমান ফাংশন থেকে তৈরি। সুতরাং, এটি আসলে ফাংশনের জন্য frame কে point করে। এখন এই ফাংশনটির execution এবং return সম্পন্ন হওয়ার পর, previous frame pointer টি register bp তে load করা হয়, সেইজন্য frame pointer তখন এখানে নির্দেশ করবে যে ফাংশন main এর সংশ্লিষ্ট। এখন stack pointer, অন্য দিকে, bottom of stack কে point করে।

(স্লাইড সময় : 08:37 দেখুন)

এখন, এটিকে আরও বিস্তারিতভাবে দেখি। আসুন আমরা বলি এটি এই stack এবং এটি বিভিন্ন stack location গুলির address এবং এটি সেই নির্দিষ্ট address এ stored data। সুতরাং আসুন আমরা অনুমান করি যে stack এর top 1000 এবং এটি নিচের দিকে হ্রাস করে। সুতরাং, এটি ফাংশনের অনুরূপ stack যখন এটি called হয়।

সুতরাং, আমরা প্রথমে দেখি যে ফাংশনটিতে pass পাস করা parameter গুলি stack এ push করা হয়; এইটি parameter গুলি 3, 2 এবং 1 যা stack এ push করা হয়। সুতরাং, এই ফাংশন parameter গুলি integer হিসাবে সংজ্ঞায়িত করা হয়েছে, তাই প্রতিটিকে 4 byte দেওয়া হয়। তাই ফাংশনটিতে pass করা integer 'a' address 997 location এ শুরু করে এবং সেখানে 4 byte 997, 998, 999 এবং 1000 থাকে। একইভাবে দ্বিতীয় ও তৃতীয় parameter গুলিও 4 byte গ্রহণ করে। এই ফাংশনের জন্য return address টি মূলত যে বিন্দুতে ফাংশনটি return করে সেটিকেও দেওয়া হয়েছে 4 byte। যেহেতু এটি একটি 32 bit সিস্টেম base pointer 4 byte দেওয়া হয়, তারপর আমরা buffer 1 যা ফাংশনের একটি local হিসাবে allocate করা হয়, যা 5 byte 976 থেকে 980 দেওয়া হয় এবং তারপর buffer 2 কে 10 byte বরাদ্দ করা হয়। সুতরাং, এই দুটি array ফাংশনের local এই বিশেষ location এ base pointer point করে এবং stack pointer এখানে address number 964 এ point করে।

(স্লাইড সময় : 10:42 দেখুন)

এখন, আসুন কিছুটা সহজ ঘটনা দেখি। তাই আসুন আমরা যদি এই নির্দিষ্ট লাইনটি print করি তাহলে কি হবে তা বলুন, printf percent x buffer 2. সুতরাং, আমরা জানি যে buffer 2 এই বিশেষ array এর address অনুরূপ, তাই এই বিশেষ printf স্টেটমেন্টটি buffer 2 এর address print করবে। সুতরাং, যদি আমরা stack টি সন্ধান করি তবে আমরা দেখতে পাই যে buffer 2 এর address শুরু 966 থেকে; সুতরাং, এই printf statement 966 print করবে। এখন আমরা যদি printf ampersand buffer 2 of 10 করি তাহলে কি হবে? আমরা জানি যে buffer 2 টি 10 বাইটের এবং 0 থেকে 9 পর্যন্ত index থাকবে; এখন buffer2 [10] হয় 966 প্লাস 10 যা 976। সুতরাং, এখানে print করা যাচ্ছে 976। এখন এটা তাই 976 buffer 2 এর অঞ্চলের বাইরে, আসলে 976 buffer 1 এ আছে। তাই, এখন আমরা যা পাচ্ছি তা হলো আমরা একটি address print করছি যা buffer 2 এর বাইরে এবং এটি buffer overflow নামে পরিচিত। মূলত, আমরা 10 বাইটের একটি buffer সংজ্ঞায়িত করেছি, তবে আমরা buffer 2 এলাকার বাইরে থাকা data access করছি। সুতরাং, আমরা 10th, 11th, 12th, ইত্যাদি বাইট access করছি। সুতরাং, এটি buffer overflow হিসাবে পরিচিত হয়। এখন, আমরা পরেরটি দেখতে পাব কিভাবে একটি attacker দ্বারা এই buffer overflow এর সুযোগ নেওয়া হতে পারে এবং কিভাবে একটি attacker সিস্টেমকে নিজের কোডটি execute করার জন্য বাধ্য করে।

(স্লাইড সময় : 12:39 দেখুন)

এখন attacker এর দৃষ্টিকোণ থেকে একটি গুরুত্বপূর্ণ জিনিস হল return address। Attacker কোনরকমে buffer 2 পূরণ করতে পারে তাহলে একটি buffer overflow হতে পারে এবং এই নির্দিষ্ট return address টি পরিবর্তন করতে পারে, তাহলে দেখা যাক কি হবে। সুতরাং আসুন আমরা বলি সে এই বিশেষ statement টি করে। সুতরাং,

buffer 2 of 19 একটি যে কোন memory location । সুতরাং, attacker কি করছে যে সে এই buffer2 টিতে overflow হতে বাধ্য করছে এবং সে এইভাবে overflow হচ্ছে যে stack এর মধ্যে সংরক্ষিত return address টি তার নিজের দেওয়া location দিয়ে প্রতিস্থাপিত হবে। ফাংশন execution সমাপ্তির পরে, এটি এই location টি দেখবে এবং বৈধ return address পাওয়ার পরিবর্তে এটি এই arbitrary location টি পাবে এবং তারপর এটি এই arbitrary location এ গিয়ে কোডটি execute করা শুরু করবে। সুতরাং, আমরা কি দেখি যে প্রত্যাশিত স্বাভাবিক প্রোগ্রামে main ফাংশন ফিরে আসার পরিবর্তে, যেহেতু attacker কিছু arbitrary location এ এই return address টি পরিবর্তন করেছে, তবে প্রসেসর এই arbitrary location এর instructions execute করবে। সুতরাং, এখন, এটি সুস্পষ্টভাবে দেখায় যে attacker কীভাবে একটি আক্রমণ তৈরি করতে পারে।

(স্লাইড সময় : 14:19 দেখুন)

মূলত attacker কি করতে যাচ্ছেন তা হল যে তিনি stack এ উপস্থিত এই বৈধ return address টিকে পরিবর্তন করে একটি attack code এর pointer রাখে। অতএব, ফাংশন যখন return করে, standard return address গ্রহণের পরিবর্তে এটি attacker code বা exploit code এর pointer নেয়; এবং ফলস্বরূপ, এটি attacker এর code চালানো শুরু করবে। এখন, আমরা দেখব কীভাবে attacker return address পরিবর্তন করে নিজের code pointer দিতে পারে।

(স্লাইড সময় : 14:56 দেখুন)

এটি করার জন্য, আমরা কি অনুমান করি যে attacker এর এই বিশেষ buffer এ access আছে। সুতরাং, এর মানে হল যে attacker এই নির্দিষ্ট buffer কে প্রয়োজনীয়ভাবে পূরণ করতে সক্ষম হবে। উদাহরণস্বরূপ, এই buffer একটি system call এর মাধ্যমে pass হতে পারে, উদাহরণস্বরূপ, যদি অপারেটিং সিস্টেমের প্রয়োজন হয় যে একটি character string user দ্বারা system call মাধ্যমে pass করা হবে, এবং এইভাবে attacker যে character string পূরণ করতে সক্ষম হবে এবং কার্নেলে এটি পাস করবে। সুতরাং, attacker কি করতে যাচ্ছেন তা হল একটি খুব নির্দিষ্ট string তৈরি করে যা এই নিজের exploit কোডটি ব্যবহার করে, এবং এটি অপারেটিং সিস্টেমকে বা কোনও অ্যাপ্লিকেশনকে বাধ্য করতে পারে এই exploit কোডটি execute করতে। সুতরাং, এটি নিম্নরূপ ভাবে এটি করতে যাচ্ছে। সুতরাং, buffer এ attacker কে দুটি জিনিস করতে হবে; প্রথমে তিনি exploit কোডটি লিখবেন, সেই কোডটি হল যে attacker buffer এর নিম্নতম অঞ্চলে execute করতে চায়, এবং তারপর buffer overflow করতে শুরু করে। মূলত তিনি এই address location BA set করবেন । এখন BA এখানে এই exploit code address। সুতরাং, এটি অনুমিত হয় বা একটি smart attacker buffer এর address location নির্ধারণ করতে সক্ষম হয়, এবং তিনি BA দ্বারা buffer overflow করবেন। তাই, তিনি buffer কে BA দ্বারা overflow করতে থাকবেন এবং যখন এটি একটি বিশেষ ক্ষেত্রে ঘটবে তখন stack এর return

address টি বৈধ return address থেকে BA দ্বারা পরিবর্তিত হয়। সুতরাং, ফাংশনটি যখন return করে তখন CPU দেখবে return address BA। সুতরাং এটি এই ঠিকানা BA নেবে এবং এটি থেকে কোড চালানো শুরু করবে। তাই, যেহেতু BA address টি সংযোজন করা হয় যেখানে কোডটি ব্যবহার করা হয় সেক্ষেত্রে CPUটি চালানো শুরু করলে এই কোডটি ব্যবহার করা হয় এবং এইভাবে attacker exploit কোডটি CPU বা processor কে execute করতে বাধ্য করে।

(স্লাইড সময় : 17:33 দেখুন)

এখন, আমরা দেখব কিভাবে একটি বিশেষ attack code কোড তৈরি করা হয় কীভাবে একটি attacker কোডটি ব্যবহার করার জন্য একটি অ্যাপ্লিকেশন বা অপারেটিং সিস্টেমকে বাধ্য করতে পারে। সুতরাং আসুন আমরা একটি exploit code এর খুব সহজ উদাহরণ গ্রহণ করি, যা এখানে দেখানো হয়েছে। মূলত এই exploit code যাকে আমরা shell code বলি কিছুই করে না শুধুমাত্র একটি বিশেষ shell চালায়, shell টি slash bin slash sh দ্বারা নির্দিষ্ট করা হয়। এবং এই বিশেষ ফাংশন execve invoke করা হয় এই shell execute করার জন্য। Parameter হল name 0 যা executable এর name সূচিত করে name 1 যা মূলত একটি null terminated string। সুতরাং, আমরা দেখব কিভাবে এই বিশেষ কোড একটি unauthorized attacker দ্বারা সঞ্চালন করতে বাধ্য করা যেতে পারে। তাই যে প্রথম প্রশ্নটি জিজ্ঞাসা করা প্রয়োজন তা হল কিভাবে এই attacker stack এ এই কোড push করে?

(স্লাইড সময় : 18:48 দেখুন)

এটি করার প্রথম ধাপ হলো attacker কে এই বিশেষ প্রোগ্রামের সাথে সংশ্লিষ্ট binary data সংগ্রহ করতে হবে। এটি করার জন্য, attacker এই কোডটি পুনরায় assembly code এ প্রোগ্রামটি লেখে। এই assembly code program যা করে ঠিক তাই করে। এরপর attacker এই assembly code compile করে এবং object dump পায়। সুতরাং এই বিশেষ কমান্ড run করলে object dump পাওয়া যায়। সুতরাং, তিনি প্রথমে এই নির্দিষ্ট কোড compile করে shell code dot o নামে object file পাবেন, এবং তারপর তিনি এই বিশেষ কমান্ড objdump disassemble চালাবেন যা সমস্ত shellcode dot o থেকে এই ফাইলটি পাওয়া যাবে। এখন, আমাদের জন্য কি গুরুত্বপূর্ণ এই বিশেষ column বা দ্বিতীয় column। তাই আপনি এখানে যে সংখ্যা রয়েছে সেগুলি hexadecimal সংখ্যা আসলে এই প্রোগ্রামের machine code । তাই যে সংখ্যাগুলি আছে যেমন eb 1e, 5e, 89 76 08, এবং তাই এই বিশেষ প্রোগ্রামের machine code । অন্য কথায়, যদি attacker এই machine code টি stack এ রাখে এবং এই machine code টি execute করতে সক্ষম হয়, তাহলে attacker প্রয়োজনীয় shell টি চালাতে সক্ষম হবে। তাই machine code এখানে দেখানো হয় এবং এই বিশেষ attack এর জন্য প্রয়োজন এমন একটি জিনিস এই

machine code এ উপস্থিত সমস্ত zero অন্য instruction দিয়ে প্রতিস্থাপন করা হয়, যাতে এখানে কোনও শূন্য উপস্থিত না থাকে।

(স্লাইড সময় : 20:48 দেখুন)

পরের জিনিসটি একটি buffer overflow এর জন্য exploited হতে পারে এমন একটি location খুঁজে পেতে সমগ্র অ্যাপ্লিকেশন scan করা হয়। সুতরাং, মূলত একটি buffer overflow এর জন্য প্রয়োজনীয় হল attacker অ্যাপ্লিকেশন কোডে একটি কমান্ড যেমন একটি string copy buffer comma large string খুঁজে বের করা যেখানে buffer একটি ছোট array এবং stack locally সংজ্ঞায়িত করা হয়, যেখানে large string একটি অনেক বড় array। সুতরাং, আমরা এই বিশেষ ফাংশন strcpy এর কাজ জানি যে, বড় string buffer এ copy করা হয় এবং এই copy byte by byte চলবে যতক্ষণ না একটি slash 0 পাওয়া যায়; সেই ক্ষেত্রে strcpy সম্পূর্ণভাবে সম্পন্ন হবে এবং return করবে। সুতরাং আসুন আমরা ধরতে পারি যে attacker এমন একটি কেস পেয়েছে যেখানে একটি buffer আছে, একটি বড় string আছে এবং একটি string copy হচ্ছে, এবং buffer একটি stack এর উপরে সংজ্ঞায়িত একটি ছোট array এবং attacker কীভাবে এটি ব্যবহার করে।

(স্লাইড সময় : 22:07 দেখুন)

সুতরাং, attacker কি করবে shell code array নামক কিছু তৈরি করবে যা মূলত যে code টি তিনি execute করতে চান সেটি। সুতরাং, তিনি সমস্ত assembly op code বা machine code নিয়ে shell code array তৈরি করেন যা তিনি execute করতে চান এটি চালাতে চায় এবং তিনি এই shell code টি বড় string এর প্রথম অংশে রাখেন। সুতরাং, যদি এই বড় string array দেখেন, যা এই ক্ষেত্রে 128 বাইটের একটি খুব বড় string, তার প্রথম অংশে shell code রাখা হয়।

(স্লাইড সময় : 22:46 দেখুন)

তারপর তিনি buffer address কী হবে তা হিসেব করেন এবং বড় string এর বাকি অংশ buffer address দ্বারা পূরণ করেন।

(স্লাইড সময় : 23:00 দেখুন)

এখন তিনি এই buffer এ string copy execute করার জন্য জোর করবেন যা এখানে দেখানো হয়েছে। এখন এই string copy টি execute করার ফলে, string copy এর জন্য একটি stack frame তৈরি হবে এবং আমরা জানি যে string copy টি বড় string থেকে buffer এ byte by byte copy করে যতক্ষণ না slash 0 পায়। সুতরাং, এইভাবে stack এর উপর বড় string টি copy করা হয়। তাই প্রথমে, shell code copy হয়ে যায় এবং তারপর

buffer address overflow হতে থাকে slash 0 না পাওয়া পর্যন্ত এটি চালিয়ে যায়। সুতরাং, যখন string copy টি আমরা যা দেখেছি তা execute করে তা হল বৈধ return address এর পরিবর্তে, এখন এটি buffer address হিসাবে পরিচিত হয়। সুতরাং, মূলত আমরা জানি যে buffer address টি এই বিশেষ location এ নির্দেশ করে, এবং সেইজন্য, CPU- বাধ্য হয় এই location এ shell code execute করতে। ফলস্বরূপ attacker shell code টি চালাতে সক্ষম হবে যা এই বিশেষ উদাহরণে exploit code ।

(স্লাইড সময় : 24:22 দেখুন)

তাই আসুন আমরা একসঙ্গে সমগ্র জিনিসটি দেখি। সুতরাং shell code আছে যা attacker execute করতে চায়। এখানে, আমরা এটি একটি global array হিসাবে সংজ্ঞায়িত করেছি, কিন্তু প্রকৃতপক্ষে এটি scanf এর মত বিভিন্ন জিনিসের মাধ্যমে প্রবেশ করা যেতে পারে বা এটি নেটওয়ার্ক কার্ডের মাধ্যমেও আসতে পারে, মূলত আমরা একটি বিশেষ format এর একটি packet pass করেছি যার মধ্যে exploit টি আছে এবং অন্যান্য বিভিন্ন উপায়ে shell code pass করা যায়। পরবর্তী, আমরা অনুমান করি যে কোথাও কোনও অ্যাপ্লিকেশনে এই বিশেষ কোড আছে। আমাদের বড় string আছে এবং একটি সংক্ষিপ্ত string আছে যেটি buffer, যা একটি local array স্থানীয়ভাবে সংজ্ঞায়িত array যা stack এর উপরে তৈরি করা হয়। সুতরাং, প্রথম কি করতে হয় একরকম buffer address এর সঙ্গে দীর্ঘ string বা বড় string পূরণ করতে পরিচালনা করা হয়। সুতরাং, আপনি মনে করে দেখুন যে, এই BA অংশগুলি উপস্থিত রয়েছে, তারপরে আমরা shell code টি বড় string এ copy করব। সুতরাং, আমরা এই বৃহৎ string টি তৈরি করেছি যা আমাদের প্রয়োজন। এই বড় string এর প্রথম অংশে shell code থাকে, এবং তারপর এটি buffer return address আছে। এবং তারপর যদি string copy এর মত একটি ফাংশন থাকে যা বড় string কে buffer এর মধ্যে copy করে তবে এর ফলে স্ট্রিং কপি করার পরে ফাংশনটি এই নির্দিষ্ট পয়েন্টে return করার পরিবর্তে একটি buffer overflow হবে। অন্য দিকে, এই বিশেষ shell code টি চালান এবং এই কম্যান্ড দ্বারা নির্দিষ্ট shell execute হয়। সুতরাং, যদি আমরা আসলে এটি দেখতে পাই, আমরা যদি gcc overflow dot c execute করি তবে এইটিকে overflow1 dot c বলা হয় এবং dot slash a dot out execute করি। শুধু এই string copy করে প্রশ্ন করার পরিবর্তে, এই বিশেষ প্রোগ্রামটি একটি নতুন shell তৈরি করেছে যেহেতু exploit code টি execute হয়।

(স্লাইড সময় : 26:44 দেখুন)

সুতরাং, গত এক দশকে বা তারও বেশি ধরে buffer over অত্যন্ত সুপরিচিত bug বাগ এবং বেশিরভাগ malware ও virus দ্বারা ব্যাপকভাবে exploited । প্রথম virus যা আসলে buffer overflow ব্যবহার করে, তা হল CODERED নামক worm যা জুলাই 13, 2001 তারিখে প্রকাশিত হয়েছিল। সুতরাং, এটি সারা বিশ্বে ব্যাপকভাবে বিস্তৃত সৃষ্টি করেছে এবং লাল স্পটগুলি আসলে দেখায় যে এই virus টি সারা বিশ্ব জুড়ে কিভাবে

ছড়িয়েছিল একদিনের মধ্যে অথবা কয়েক ঘণ্টার মধ্যে। তাই, আমরা দেখি যে এই বিশেষ virus টি যা buffer overflow ব্যবহার করে প্রায় 19 শে জুন 2001 তারিখের মধ্যে 359000 computer কে আক্রান্ত করে।

(স্লাইড সময় : 27:36 দেখুন)

সুতরাং মূলত এই worm এর লক্ষ্য ছিল Microsoft এর IIS web server এবং যে string execute করা হয়েছিল তা এখানে দেখানো হল। সুতরাং, এই string প্রকৃতপক্ষে কোডটি ব্যবহার করা হয়েছিল যা execute করা হয়েছিল এবং এর ফলে web browser এ এইরকম কিছু প্রদর্শিত হয়েছিল। ধন্যবাদ।